김민성 Backend Engineer

mskim.bm@gmail.com 010-8976-2340 github.com/mskim98

Summary

한정된 자원을 효율적으로 사용하며 측정치를 근거로 문제에 접근하고 테스트로 검증하는 **JAVA/Spring** 백엔드 개발자입니다.
비동기 처리, 고가용성, 안정성, 병목 개선에 강점을 가지고 있습니다.

- 커머스, LLM, ERP, 위치 기반 서비스 등 다양한 도메인에서 백엔드 시스템을 설계하고 개발한 경험이 있습니다.
- AI로 코드 품질을 검증하며 요구사항 을 충족하는 서비스를 구현한 경험이 있습니다. 직접 구현한 코드는 AI를 코드리뷰어, 검증기, 보고서 생성기로 역할 분리한 Harness로 로직/설계 검토와 테스트 검증, 품질 보고서 정리를 수행한 경험이 있습니다.
- 정합성, 멱등성이 중요한 결제 기능, 고가용성을 위한 Redis Cluster 구축, 자원 효율성을 위한 스레드 풀 튜닝, 장애 전파 방지를 위한 서킷브레이커를 구현해보았습니다.

Project

AI 튜터링 서비스(Opener) 백엔드 리드 · 9인 팀(백엔드 2인) · 2025.12 ~ 2026.03 GitHub
학생이 수능·모의고사 문제를 질문하고 실시간 AI 튜터링을 받는 교육 플랫폼
기술 | Spring, Spring AI, Reactor, PostgreSQL/pgvector, Redis, RabbitMQ, Resilience4j

실시간 채팅 스트리밍
학생이 문제를 질문하면 LLM 응답을 SSE로 글자 단위로 반환하는 튜터링 채팅 기능

- 토큰 서빙을 LLM 응답 스트림에 논블로킹 방식의 Reactive flux를 구독하여 응답을 기다리는 동안 요청 스레드를 점유하지 않게 하고, 토큰마다 거치는 Redis 저장 같은 블로킹 I/O는 오프로딩으로 별도 스레드에서 비동기 처리해 이벤트 루프가 다른 세션을 막지 않게 설계했습니다.[Blog: 추가예정]
- 호출 성격이 다른 LLM과 Redis에 별도 서킷브레이커 정책을 두어 한쪽 장애가 반대쪽 경로를 막지 않게 설계했습니다.

채팅 세션 저장소
진행 중인 대화를 Redis에 버퍼링해 멀티턴 문맥으로 쓰고, 세션이 끝나면 PostgreSQL에 영구 저장하는 기능

- 채팅 세션 저장소를 Master 3 / Replica 3의 Redis Cluster로 구성하고 같은 세션의 키로 동일 슬롯에 할당하였고 인증용 저장소는 분리해 장애가 전파되지 않게 설계했습니다.
- 멀티턴 채팅 이력의 쓰기는 Master, 조회는 Replica로 라우팅하는 방식으로 분리하고 조회는 다시 프롬프트용 최근 N턴과 요약, 영속화용으로 나눴습니다. 쓰기 직후 읽는 경로만 Master로 고정해 복제 지연에 의한 누락을 방지했습니다.
  합계 **107,238 ops/s**, 쓰기 p95 **1.34ms**, 응답 p95 **133ms**
- 대화 이력 저장 시 RabbitMQ를 통해 비동기로 이벤트를 발행하여 DB에 적재하도록 하고, 컨슈머가 저장시점의 데이터 상태에 따라 유실, 재시도 실패 이벤트만 DLQ로 후속처리할 수 있게 적재하였습니다.

RAG 검색 파이프라인
질문과 유사한 수학 모의고사 문제·풀이를 pgvector를 통해 유사문제 생성, 제공 기능

- HNSW 인덱스 기반 Vector DB를 이용하여 수학 문제, 풀이를 평균 **361ms**로 검색하여 LLM에 질의하는 RAG을 구축하였고 적재된 데이터가 없는 경우 Fallback 파이프라인으로 분리하여 검색 실패가 응답 실패로 이어지지 않게 설계했습니다.
- 질문 20개에 정답 분류를 붙여 두고 상위 5건 안에 정답 분류 문서가 들어오는 질문의 정답율로 정확도를 측정하여 RAG 도입을 통한 문서 서칭 정확도 향상을 측정했습니다. Hit Rate@5 **85%**(17/20)
- 그림이 Asymptote 소스 텍스트로 적재된 청크(787개, 13.6%)가 유사도를 낮추는 것을 찾아 색인 대상에서 제외했습니다.

인증/세션, 인프라, 관측
폼·카카오 OAuth2 로그인과 JWT 세션, 컨테이너 배포 파이프라인과 지표 수집

- 로그아웃 시 access token을 Redis 블랙리스트에 남은 수명만큼 등록하고 refresh 토큰을 함께 폐기하며, Redis 장애 시에는 인증을 거부하는 fail-close로 구성했습니다.
- Multi-stage Dockerfile 이미지를 GitHub Actions에서 레지스트리 캐시로 빌드해 EC2에 자동 배포하고, SSE 전용 Nginx location에 proxy_buffering off를 두어 청크를 즉시 전달하게 구성했고,
  Actuator + Prometheus + Grafana로 처리량·지연·서킷 상태를 관측하도록 구성했습니다.

이커머스 백엔드(groom-shopping) 팀장 · 4인 팀 · 2025.10 ~ 2025.12 GitHub
상품을 주문·결제하고 쿠폰과 추첨 이벤트를 운영하는 이커머스 백엔드
STACK | Spring, JPA, PostgreSQL, Redis, Kafka, Resilience4j

한정 수량 쿠폰 파이프라인
정해진 수량의 쿠폰을 동시에 몰리는 요청에 발급하고, 기간이 끝나면 자동으로 비활성화하는 기능

- 쿠폰 이벤트 발행시 Redis Lua를 이용한 선제적 판별 이후 사용자 ID로 해싱하여 여러 파티션에 나누어 적재하고 컨슈머들이 병렬 처리할 수 있게 설계하여 대규모 요청을 감당할 수 있게 구현했습니다..
  (조건 : rps 2,000, 총 10만 건에서 재고) 발급 완료 p95 **118ms, 오버셀링 없음**
- 발급 가능 여부는 중복 검사, 재고 차감, 발급자 등록을 Redis Lua로 원자적으로 처리, I/O작업을 분리해 즉시 반환하고 Kafka로 이벤트를 발행하여 비동기로 처리하였습니다. 직접적인 DB 접근을 한정하여 커넥션 풀의 고갈 가능성과 서버부하를 낮추었습니다.
- 특정 쿠폰 재고에 UPDATE가 집중되는 Hot Row 문제를 발행 테이블의 INSERT 구조로 전환하여 Row Lock을 제거하고, 동시 요청 간 쓰기 경합을 완화했습니다.

PG 결제 보상 파이프라인
PG 결제 승인 뒤 주문 확정 같은 DB 후처리가 실패하면 승인된 금액을 자동으로 취소해 돈과 장부를 맞추는 기능

- 비가역적인 PG 승인과 가역적인 DB 후처리를 한 트랜잭션에 묶지 않고, 승인은 트랜잭션 밖에서 처리고 후처리는 독립 트랜잭션으로 분리하여 후처리 실패를 보상 트랜잭션으로 되돌리도록 설계했습니다.
- 결제 후속 처리를 Outbox 패턴으로 별도의 결제 테이블에 적재 후 Kafka로 이벤트를 발행하여 이벤트 큐 장애시 유실을 방지하고 재처리 할 수 있게 설계했습니다.
- 결제를 상태 머신으로 추적,관리하여 정상적이지 않는 경로로 동작하지 못하게 설계했습니다.
- 결제 완료 건을 재시도하지 못하게 하고, 취소 멱등키를 고정해 응답만 유실된 재시도가 이중 취소가 되지 않게 설계했습니다.
  실패는 지수 백오프로 미루고 한도 초과 시 DLQ로 넘겨 후속 처리할 수 있게 했습니다.
- 승인과 취소 서킷을 분리해 승인 장애가 취소 호출까지 차단하지 않게 하고, 배치가 한 페이지로 대기를 비우지 못하면 취소 회로를 열고 회로 OPEN 시간을 배치 주기보다 길게 두어PG 장애 구간에는 다음 회차를 생략하도록 설계했습니다.
  장애 주입 실측: 취소 장애 중 p95 **310ms**, 승인 장애 중 fast-fail p95 **649.2ms**, PG 미도달 차단 **97.8%**

주문 시 상품 재고를 차감하고 동시 주문에서 초과 판매를 막는 기능

- 낙관적 락, 조건부 UPDATE, 비관적 락을 같은 부하로 비교해 비관적 락을 기본 전략으로 두고, 주문 진입 시 Redis 선점 게이트로 주문 시점 검증이 못 덮는 경쟁 구간을 DB 도달 전에 차단하게 설계했습니다. 커넥션 풀 크기는 부하 실측으로 정했습니다. 주문 1만 건 기준 성공률 **100%,** p95 **100.9ms**, **오버셀링 0**

휠 파인더(Wheel Finder) 팀장 · 4인 팀 · 2025.09 ~ 2025.10 GitHub
전동 휠체어·스쿠터 사용자를 위한 카카오 지도 기반 충전소 정보·경로 안내 플랫폼
기술 | Spring Boot, JPA, PostgreSQL, Redis, OAuth2, SSE, Next.js, Nginx, Docker

보호자 실시간 위치, 경로 공유
보호자가 전동보장구 사용자의 현재 위치와 안내 중인 경로를 실시간으로 보는 기능

- 위치 이벤트를 Redis Pub/Sub으로 발행하고 각 인스턴스가 구독해 자기 SSE 연결에 전달하도록 구성하여 보호자가 어느 인스턴스에 연결돼도 위치가 도달하게 설계했습니다. 경로는 Redis Stream에 남겨 발행 뒤에 접속한 보호자도 안내 중인 경로를 조회할 수 있게 했습니다.
- 연결 생명주기 콜백으로 끊어진 커넥션을 회수하고 재연결 시 기존 연결을 교체해 같은 관계에 커넥션 1개만 유지하도록 설계했습니다.

배포, 개발환경 구성
Nginx·Docker 기반 환경구성

- Nginx 리버스 프록시와 Docker 인프라를 설계하고 기획안·ERD·API 명세서로 팀원 간 개발 범위와 인터페이스를 정의했습니다.

발주관리시스템 팀장 · 3인 팀 · 2025.03 ~ 2025.04
프랜차이즈 본사 품목을 전국 점포가 발주하고 마감 후 파트너에게 전달하는 시스템
기술 | NestJS, PostgreSQL, Nginx, Docker

RBAC 인증, 인가
본사·점포·파트너 역할별 권한 제어와 발주 데이터의 일관성 유지

- JWT 검증, 역할 확인, 리소스 권한 평가로 이어지는 3단계 Guard 체인을 구성했습니다.

커서 기반 페이지네이션 및 발주 자동화
발주 목록 조회와 정기 발주 자동 생성

- 발주 목록을 커서 기반 페이지네이션으로 구현해 웹·모바일 공통으로 제공합니다. 응답 시간 **10ms**
- CRON 스케줄러로 요일·시간별 발주 주기를 자동화했습니다.

Skills

Language : Java, TypeScript, JavaScript	**Framework :** Spring(Boot, MVC, WebFlux, AI), NestJS, NextJS
Database : PostgreSQL, MySQL, Redis, pgvector, JPA	**Message Queue :** Kafka, RabbitMQ
Infra : Docker, Nginx, AWS(EC2, S3, RDS), GitHub Actions	**Monitoring :** Prometheus, Grafana, Loki

Activity

구름 백엔드 부트캠프 대상	2026.01	구름(goorm) 주관
구름 백엔드 부트캠프	2025.07 ~ 2026.01	Spring 기반 백엔드 심화 과정
테커(Techeer)	2023.12 ~ 진행 중	개발자 커뮤니티 활동

Education

인천대학교 / 메카트로닉스공학과 / 2018.03 ~ 2025.02 (졸업)

BACKEND ENGINEER · PORTFOLIO

김민성

MINSUNG KIM

요청이 몰리는 구간에서 **처리량 · 스레드 자원 · 장애 복구**를 함께 지키는 서버를 설계합니다.

대용량 처리

505 → 2,000 RPS

재고 1만 개에 초당 2,000건 선착순 요청 — 초과 · 중복 발급 0, 발급 p95 32.3초 → 118ms
재고 행 잠금(505 RPS) → Redis 판정 선행 · Kafka 비동기 확정 · DB 커넥션 최대 4개(인증 제외)

고가용성

Master 장애 → 7.85초 쓰기 재개

Redis 단일 노드(단일 장애점) → 채팅 세션만 Cluster 이중화 · 장애 판정 5초 + 자동 승격
PG 승인 뒤 DB 실패 200건 → 보상 배치가 사람 개입 없이 취소(p95 98.5초)

AI 품질 검증

리뷰 · 검증 · 보고서 역할 분리

현장 요구사항 파악 · 리서치로 더 나은 개발 옵션 제시
구현 코드는 AI 하네스로 로직 · 설계 검토 → 테스트 기반 검증 → 품질 보고서 정리

Phone
010-8976-2340

Email
mskim.bm@gmail.com

GitHub
github.com/mskim98

Contents

목차

팀 프로젝트 2건 · 백엔드 리드 · 이커머스 10장(03–12) · AI 튜터링 6장(13–18) · AI 하네스(19)

이커머스 백엔드

pg.03 구조와 부하 지점
도메인 구성 · 요청이 몰리는 자리

pg.04–09 쿠폰 발급
시퀀스 · 접근 · 결과 · 튜닝 · 폴백

pg.10–12 결제
시퀀스 · 파이프라인 · 보상

AI 튜터링

pg.13 구조와 응답 경로
저장소 분리 · 스트리밍 전달 경로

pg.14–17 실시간 스트리밍
시퀀스 · 스레드 모델 · 오프로딩 · 서킷

pg.18 세션 저장소
Cluster 이중화 · 읽기 분산

개발 방식

pg.19 AI 하네스
코드 리뷰어 · 검증기 · 보고서 생성기 역할 분리

ABOUT ME

두 팀 프로젝트 모두 **백엔드 리드**로 참여해, 동시성·외부 연동·실시간 전송처럼 **실패가 데이터에 남는 경로**를 맡았습니다. 선택한 구조마다 부하 측정으로 근거를 남겼습니다.

이커머스 백엔드 · 4인 팀 팀장
(2025-10 ~ 2025-12)
AI 튜터링 · 9인 팀 백엔드 리드
(2025-12 ~ 2026-03)

03

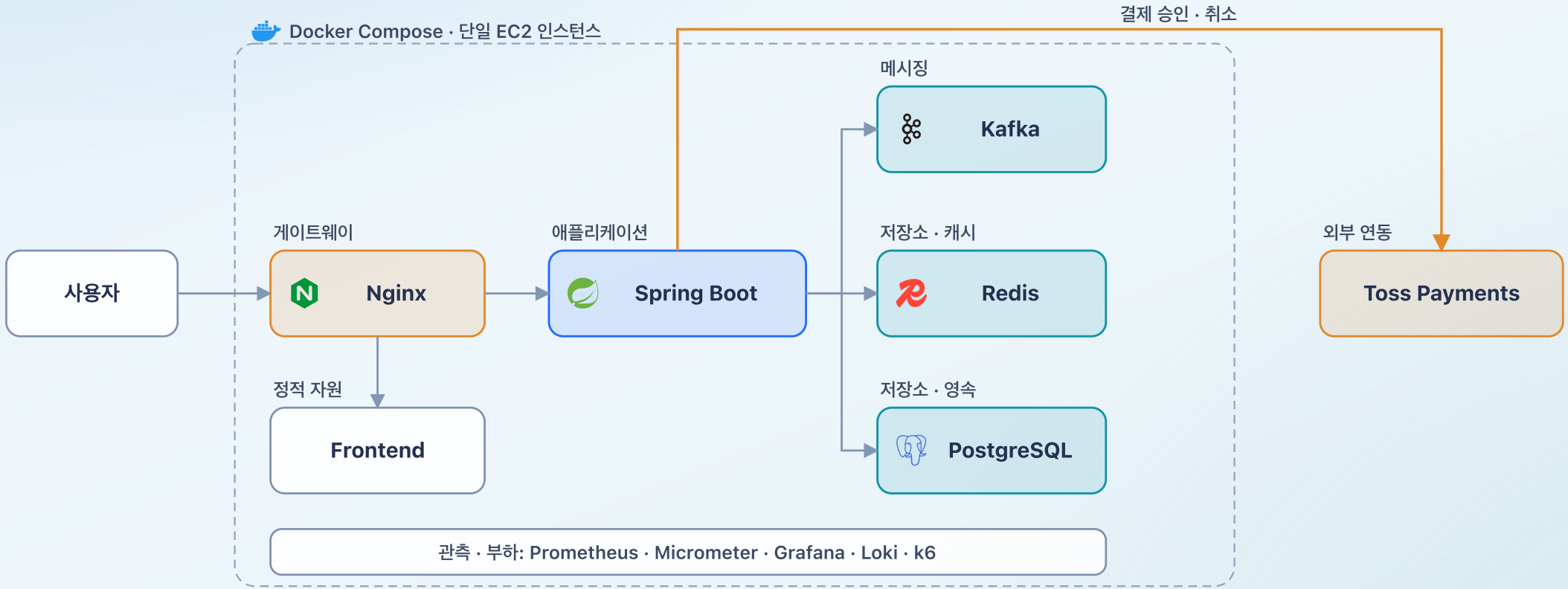
이커머스 · Overview

이커머스 백엔드

2025-10 ~ 2025-12 · 4인 팀 팀장 / 백엔드 리드

한정 수량 쿠폰 발급과 결제(승인 · 보상)를 다루는 이커머스 백엔드. **쿠폰 발급 · 결제 두 갈래**를 설계 · 구현
전 구간에서 담당 (4인 팀 팀장 · 백엔드 리드, 도메인 모델링 · 동시성 제어 · 결제 연동 · 부하 테스트 주도).

Spring · JPA · PostgreSQL · Redis · Lua · Redisson · Kafka · Resilience4j · Docker · Nginx · k6 · Grafana



김민성 · Minsung Kim

04

이커머스 · 쿠폰 발급 · Sequence

발급 한 건이 흐르는 순서

접수는 톰캣 스레드, 확정은 컨슈머 스레드 — ⑥에서 응답이 끝난다

문제 한 트랜잭션 안의 재고 행 잠금이 모든 요청을 직렬화 — 2,000 RPS 에서 p95 **32.3초**. 그래서 접수와 확정을 나눔.
⑥ 의 `202 + requestId` 로 응답이 끝나고 ⑦~⑩ 은 요청 스레드와 무관. **p95 118ms 는 ①~⑫** — 접수가 아니라 **확정 확인까지**.



김민성 · Minsung Kim

05

이커머스 · 쿠폰 발급 · 접근 방식

트랜잭션은 INSERT 한 번으로, 요청 스레드는 커넥션 없이

커넥션 풀 50 · 컨슈머 4 · 인증 제외 측정 — 쿠폰 경로의 요청 스레드는 Redis 판정과 이벤트 발행까지, DB 는 컨슈머의 INSERT 한 번만

STEP 1

판정 선행

Redis 가 재고 · 중복 발급 여부를 **선제적으로 판별**해 발급 가능한 요청만 통과 — DB 부하를 낮춤

STEP 2

락 대기는 트랜잭션 밖

분산락 대기 · Redis 되돌림은 트랜잭션 밖(**NOT_SUPPORTED**), 저장만 **REQUIRES_NEW · timeout**. 쿠폰 커넥션 50 → 1(락이 저장을 직렬화)

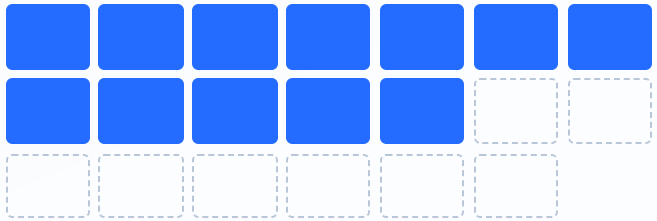
STEP 3

확정을 컨슈머로

요청 경로의 트랜잭션을 없애고, 재고 행 UPDATE 대신 **발급 행 INSERT 한 번**. 쿠폰 커넥션 1 → 4(컨슈머 병렬)

요청 스레드

가상 스레드 — 풀 크기 제한 없음, 쿠폰 경로에서 DB 커넥션 미점유



2,000 RPS 에서 동시 처리 p95 12건 · 최대 102건(1회)

→

컨슈머 4

키 userId 로 파티션 4개에 분산 · poll 한 번에 최대 500건

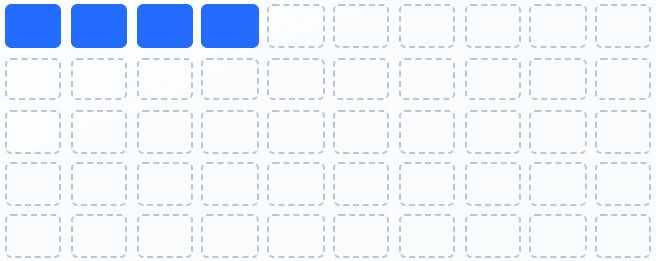


INSERT … ON CONFLICT 한 번

→

커넥션 풀 50

확정을 맡은 컨슈머만 커넥션 사용



최대 4 / 50 · 대기 0

■ 실제 작업 중 ▢ 비어 있음

트랜잭션 범위 — 쿠폰 경로에서 커넥션을 쓰는 구간은 컨슈머뿐

중복 · 재고 판정 (Redis Lua) › 상태 WAITING › Kafka 발행 › 202 응답 › 쿠폰 조회 › **INSERT … ON CONFLICT** › 상태 SUCCESS

요청 스레드 — 트랜잭션 · 커넥션 없음

컨슈머 스레드 — INSERT 만 REQUIRES_NEW · timeout 10초

06

이커머스 · 쿠폰 발급 · 결과

같은 부하에서 505 → 2,000 RPS, p95 32.3초 → 118ms

세 단계 모두 2,000 RPS × 50초 · 총 10만 건 · 재고 1만 · 사용자 9만 명(1만 건은 같은 사용자 재요청) · 인증 제외 — 한 번에 하나씩 바꾸고 다시 측정

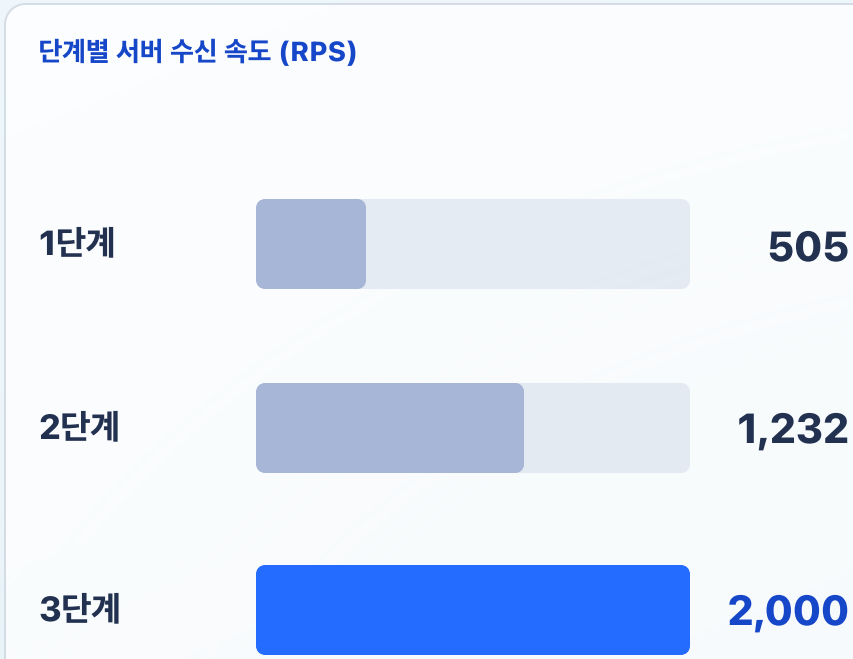
2,000 RPS
서버 수신 속도
505 RPS → 유입 전량 수신

118ms
발급 요청 p95
32.3초 → 118ms

10,000건
재고 1만 중 발급
6,676 · 460 → 10,000

4개
쿠폰 경로 커넥션 최대
풀 50 중 · 50 → 1 → 4

단계	수신 속도 (RPS)	p95	발급	커넥션	조회 성공	잠금 초과 거절
1단계 — DB 행 잠금	505	32.3초	6,676	50	20%	0
2단계 — 판정 선행 · 분산락	1,232	12.8초	460	1	100%	64,106
3단계 — 비동기 확정	**2,000**	**118ms**	**10,000**	**4**	**100%**	**0**
3단계 · 스파이크 5,000 RPS	2,100	1,647ms	10,000	4	100%	0



김민성 · Minsung Kim

07

이커머스 · 쿠폰 발급 · 튜닝

컨슈머 · 파티션 수 — 늘려도 확정이 빨라지지 않고 커넥션만 증가

파티션 = 컨슈머 4 · 8 · 16 을 같은 부하(2,000 RPS × 50초)로 지점마다 재기동 · 웜업 1회 뒤 3회, 확정 p95 가 가장 짧은 회차 기준 → 컨슈머 · 파티션 4 로 확정

39ms
컨슈머 4 확정 p95
8 → 39ms · 16 → 52.5ms

1.75 → 6.95개
평균 점유 커넥션
컨슈머 4 → 16 · 풀 50

8.2 → 2.7행
INSERT 한 번에 담긴 행
호출 1,216 → 3,641회

4 = 4
파티션 = 컨슈머
파티션당 컨슈머 1개

컨슈머	확정 p95	접수 p95	커넥션 순간 최대	평균 점유	INSERT 호출	호출당 행
4	**39ms**	**12ms**	**4**	**1.75**	**1,216**	**8.2행**
8	39ms	13ms	8	3.43	2,253	4.4행
16	52.5ms	18ms	16	6.95	3,641	2.7행

근거 1 **늘려도 빨라지지 않음** — 확정 p95(접수 → DB INSERT) 39 → 39 → 52.5ms. 컨슈머 4개가 이미 발급 10,000건을 도착 속도대로 약 5초 안에 확정해 확정 경로는 병목이 아님

근거 2 **커넥션만 증가** — 평균 점유 1.75 → 3.43 → 6.95개. 이 풀 50 은 **주문 · 결제와 함께 쓰는 풀**이고, 컨슈머가 늘수록 한 번에 받는 양이 줄어 INSERT 가 1,216 → 3,641회로 쪼개짐

선택 **컨슈머 4 · 파티션 4** — 1:1 로 맞춰 노는 컨슈머도, 파티션 여러 개를 차례로 맡는 컨슈머도 없게 배정

한계 처리 상한은 확정이 아니라 **접수 경로** — 2,500 RPS 까지 접수 p95 175ms · 미발송 0.2%, 3,000 RPS 부터 미발송 8~55%(9/2 지점당 1회)

08

이커머스 · 쿠폰 발급 · 튜닝

배치 상한 — 파티션당 1초치 500, 평소에는 닿지 않는 한도

컨슈머 · 파티션 4 · 상한(max.poll.records) 50 · 500 · 2,000 을 같은 부하로 웜업 1회 뒤 3회 → 파티션당 1초치 500 으로 확정 · 위 k6 원문은 채택 설정의 9/23 재현 회차

```
TOTAL RESULTS

  CUSTOM
  issue_accept_latency...........: avg=4.2ms     min=1.12ms   med=2.82ms max=34.78ms  p(90)=8.13ms  p(95)=11.25ms p(99)=22.24ms
  issue_dup......................: 9996    41.641431/s
  issue_soldout..................: 80000   333.264752/s
  issue_success..................: 10000   41.658094/s

  EXECUTION
  dropped_iterations.............: 4       0.016663/s

$ python3 scripts/analyze-e2e.py backend/docs/measurements/coupon_kafka_re2e-thr-r1_2026-09-23
  T_accept  (t1-t0, 접수 건): n=10000 p50=3.0 p95=12.0 p99=22.0 max=35.0 ms
  T_e2e     (촘촘한 폴링 표본만): n=500 p50=105.0 p95=118.0 p99=127.0 max=135.0 ms
  T_confirm (created_at-t1, DB): n=10000 p50=11.5 p95=38.5 p99=54.5 max=76.5 ms
```

상한	INSERT 호출 (3회)	호출당 평균	한 번 최대	상한 도달
50	4,150	7.2행	50행	3회 (0.07%)
500	**4,746**	**6.3행**	**62행**	**0**
2,000	6,408	4.7행	59행	0

근거 **2,000 RPS** 가 userId 키로 파티션 4개에 흩어져 **파티션마다 초당 약 500건** — 파티션 하나를 컨슈머 하나가 맡아 컨슈머당 초당 500건

선택 **상한 500** — 컨슈머가 1초 밀려 쌓인 500건을 poll 한 번 · INSERT 한 번으로 처리하는 크기

관측 평소 INSERT 한 번에 평균 5~7행 · 가장 많아도 62행이라 500 · 2,000 은 **한 번도 닿지 않음** — 상한이 작동한 경우는 50 의 3회(0.07%)뿐

트레이드오프 상한을 키우면 한 배치가 실패했을 때 다시 처리할 범위만 커짐

김민성 · Minsung Kim

09

이커머스 · 쿠폰 발급 · Fallback

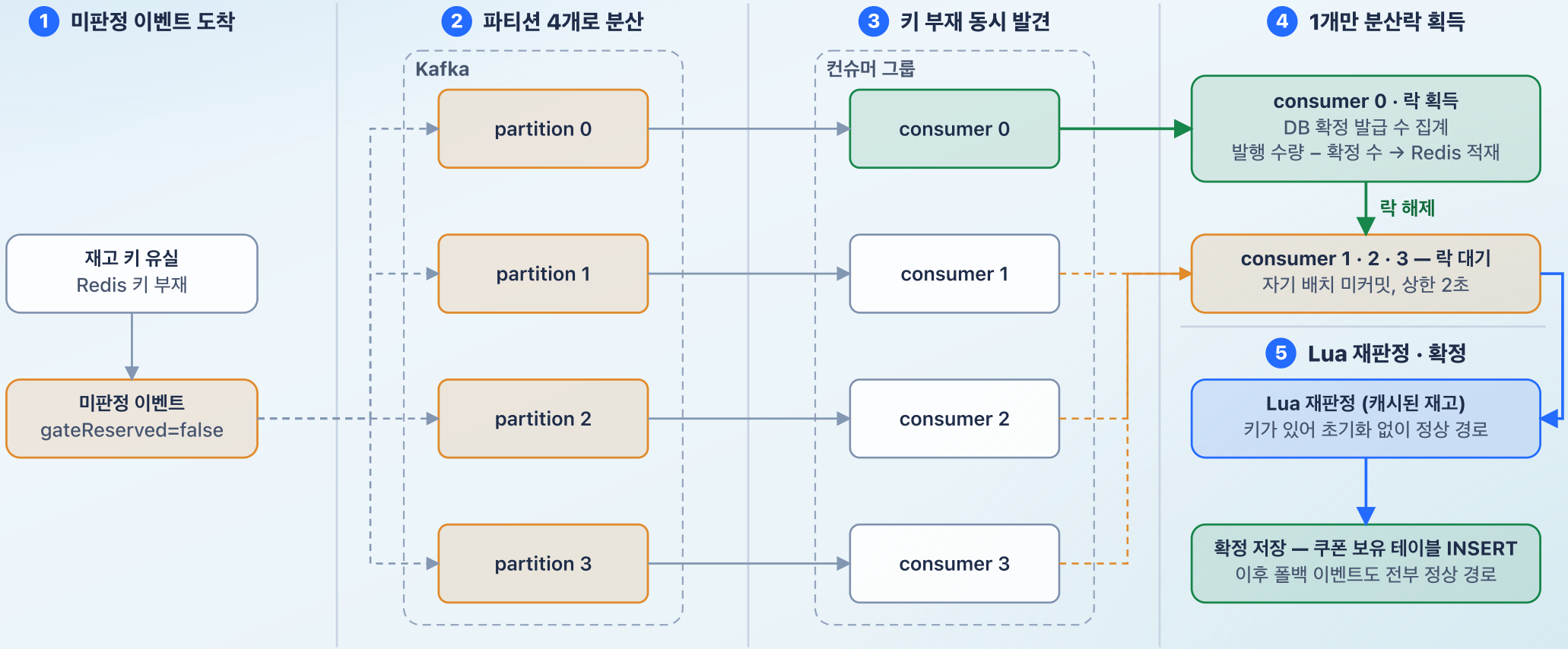
재고 키 부재 폴백 파이프라인

접수는 기다리지 않고 거절, 컨슈머는 하나만 락을 잡고 나머지는 기다렸다 재판정

문제 재고 키가 유실된 상태에서 판정 없이 실린 이벤트(`gateReserved=false`, 배포 전 큐에 남은 구 형식)는 컨슈머가 재고를 심고 다시 판정해야 함. 키가 `userId` 라 파티션 4개에 흩어져 **컨슈머 4개가 동시에 키 부재를 발견** — 그대로 두면 넷이 각자 DB 를 집계.

해결 **쿠폰 단위 분산락으로 초기화를 한 컨슈머에 맡기고** 나머지는 락이 풀린 뒤 Lua 재판정으로 합류.

결과 2,000 RPS · 1만 건 — DB 집계 **1회**, 발급 **5,000건 = 발행 수량**, **5.06초** 안에 처리(3회 동일)



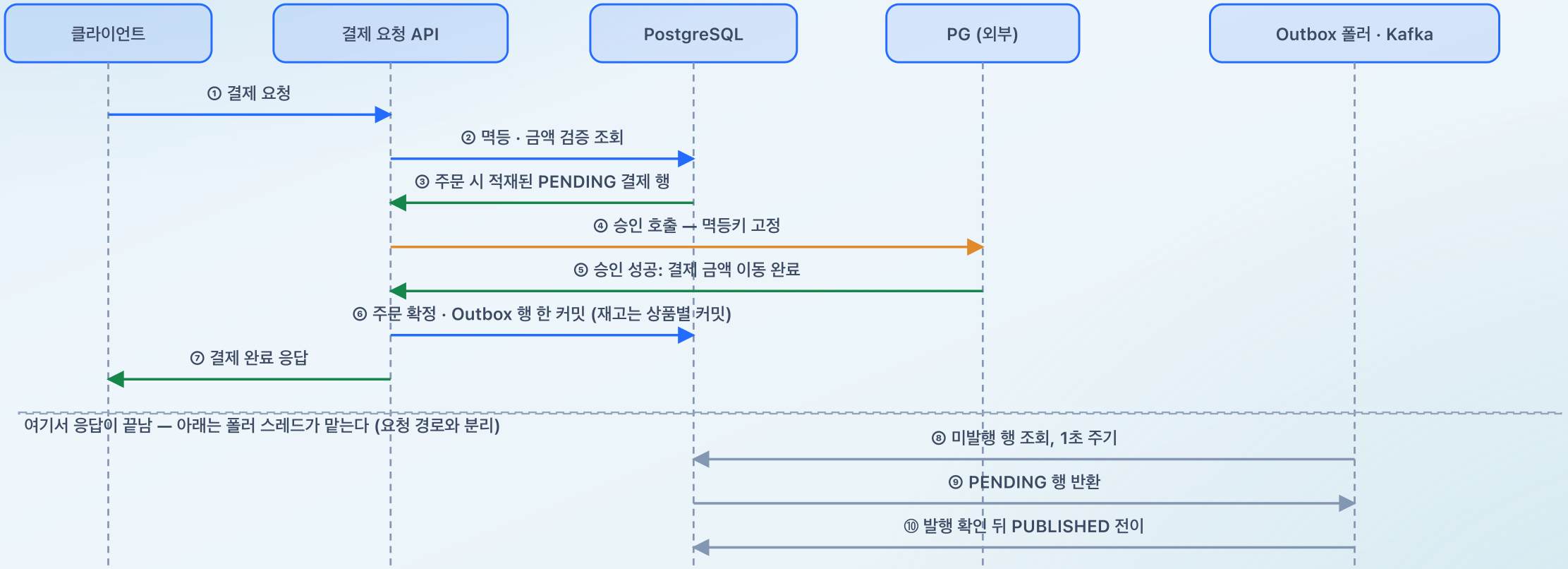
10

이커머스 · 결제 · Sequence

결제 승인 경로

승인은 트랜잭션 밖, 주문 확정 · Outbox 는 한 커밋 — 이벤트는 Outbox 폴러가 요청 경로와 분리해 발행

PG 승인은 되돌릴 수 없고 DB 후처리는 되돌릴 수 있다. 그래서 승인(④⑤)은 트랜잭션 **밖**, 주문 확정 · Outbox 적재(⑥)는 **한 커밋**(재고는 상품별 독립 커밋), 발행은 폴러(⑧~⑩)가 맡아 **롤백된 주문의 이벤트가 나가지 않음**.



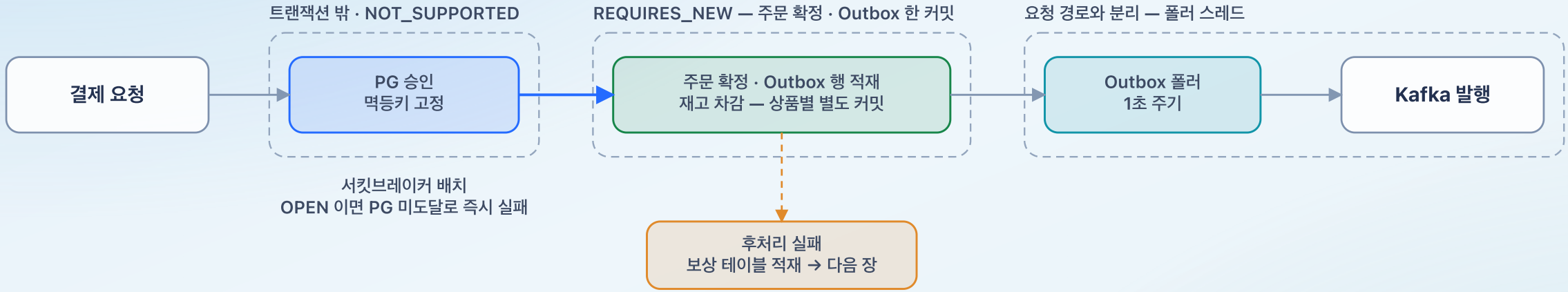
11

이커머스 · 결제 · Pipeline

결제 승인 파이프라인

트랜잭션 경계 분리와 승인 경로 서킷브레이커 — 실패가 커넥션을 붙잡지 않게

앞 장을 **트랜잭션 경계**로 나눈 그림 — 승인은 밖, 주문 확정 · Outbox 는 한 커밋, **승인 경로는 서킷브레이커 격리**.



문제 PG 승인은 되돌릴 수 없고 DB 후처리는 되돌릴 수 있는데 한 경계 안에 있어, 결제만 성립하고 주문은 미확정인 상태가 남음. PG 가 느려지면 승인 호출이 그대로 대기 — 5초 지연 주입 시 PG 에 닿은 호출 p95 **5.65초**

해결 승인은 트랜잭션 밖(NOT_SUPPORTED), 후처리는 독립 트랜잭션(REQUIRES_NEW)으로 분리. 결제완료 이벤트는 Outbox로 주문 확정과 같은 커밋에 적재, 재고 차감은 상품별 독립 커밋(도중 실패 시 복원), 서킷브레이커를 승인 · 취소 경로에 각각 배치

결과 서킷브레이커 fast-fail로 장애 격리 — 승인 경로 p95 **5.65초 → 649.2ms**. 5s 지연 + 500 주입 구간 호출 **682건** 중 **667건**을 PG 미도달로 차단, 실제 PG 에 닿은 것은 15건

5.65초
PG 에 닿은 승인 p95
서킷이 막기 전 15건 · 5초 지연 주입

649.2ms
승인 fast-fail p95
서킷브레이커 장애 격리

667 / 682건
PG 미도달로 차단
응답 1.5초 이하 = PG 미경유 (스텝이 모든 승인을 5초 지연)

김민성 · Minsung Kim

12

이커머스 · 결제 · Compensation

PG 결제 보상 파이프라인

Toss Payments 승인 이후 DB 후처리 실패를 사람 개입 없이 보상

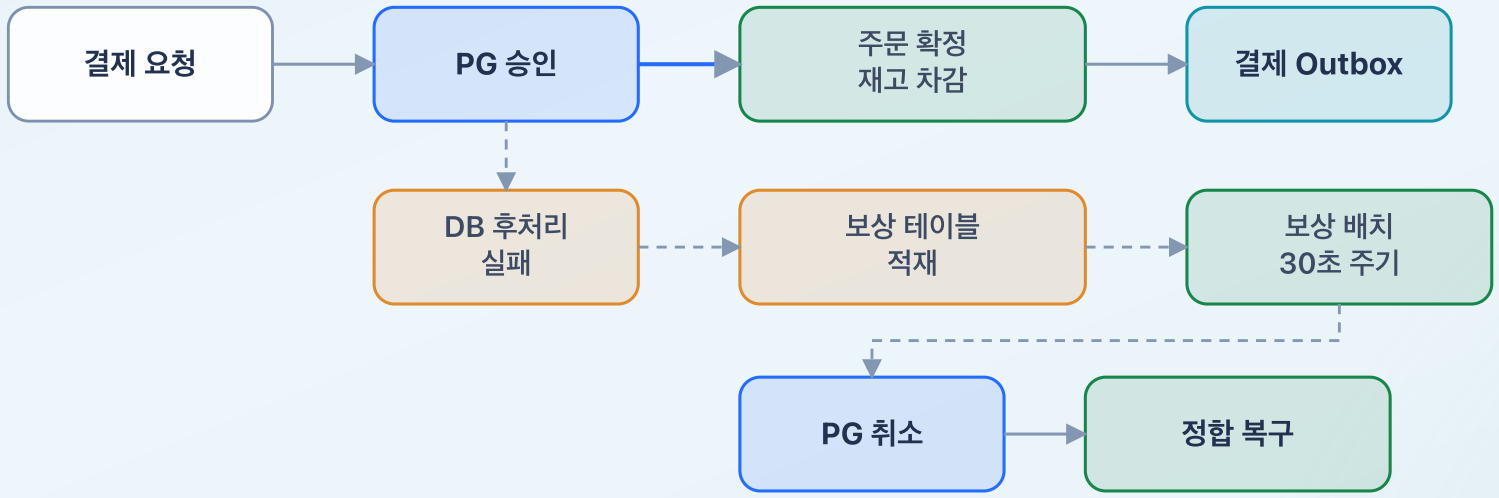
PG 승인 뒤 DB 후처리가 실패하면 **결제만 성립하고 주문은 미확정인 구간** 발생 — 보상 배치가 PG 취소까지 자동 처리.

재시도 · 백오프 · 회로 값이 서로를 참조해 정해진 근거

① 재시도 5회 · 백오프 2·4·8·16분
배치 5회가 **유일한 취소 경로**라 중간에 포기할 수 없다. 백오프 합 30분이 **하한**이고, 상한은 **PG 멱등키 유효 기간** — 그보다 길면 같은 키로 재시도해도 PG 가 받지 않는다.

② OPEN 45초 vs 배치 주기 30초
OPEN 이 주기보다 **15초 길게** 잡혀 있어, 서킷이 열린 동안 **다음 회차는 반드시 한 번 생략**된다. 감수한 비용.

③ 상태 머신 · 멱등키 고정
결제 · 보상 상태는 **상태 머신**으로 관리하고, 재시도마다 **같은 멱등키**를 써 이중 승인 · 취소를 막는다.



문제 PG 승인 뒤 DB 후처리가 실패하면 결제만 성립하고 주문은 미확정인 상태가 남음. 승인은 되돌릴 수 없어 **PG 취소**로만 정합 복구 가능

해결 실패는 보상 테이블에 적재해 상태 머신으로 관리, 30초 주기 보상 배치가 PG 취소까지 재시도(5회 · 백오프 2·4·8·16분), 취소 경로에도 서킷브레이커 배치

결과 실패 200건이 사람 개입 없이 취소 완료까지 수렴 — p95 **98.5초**. 감수한 비용: OPEN 유지 **45초**가 배치 주기 30초보다 길어, 서킷이 열린 동안의 회차는 반드시 하나 생략됨

13

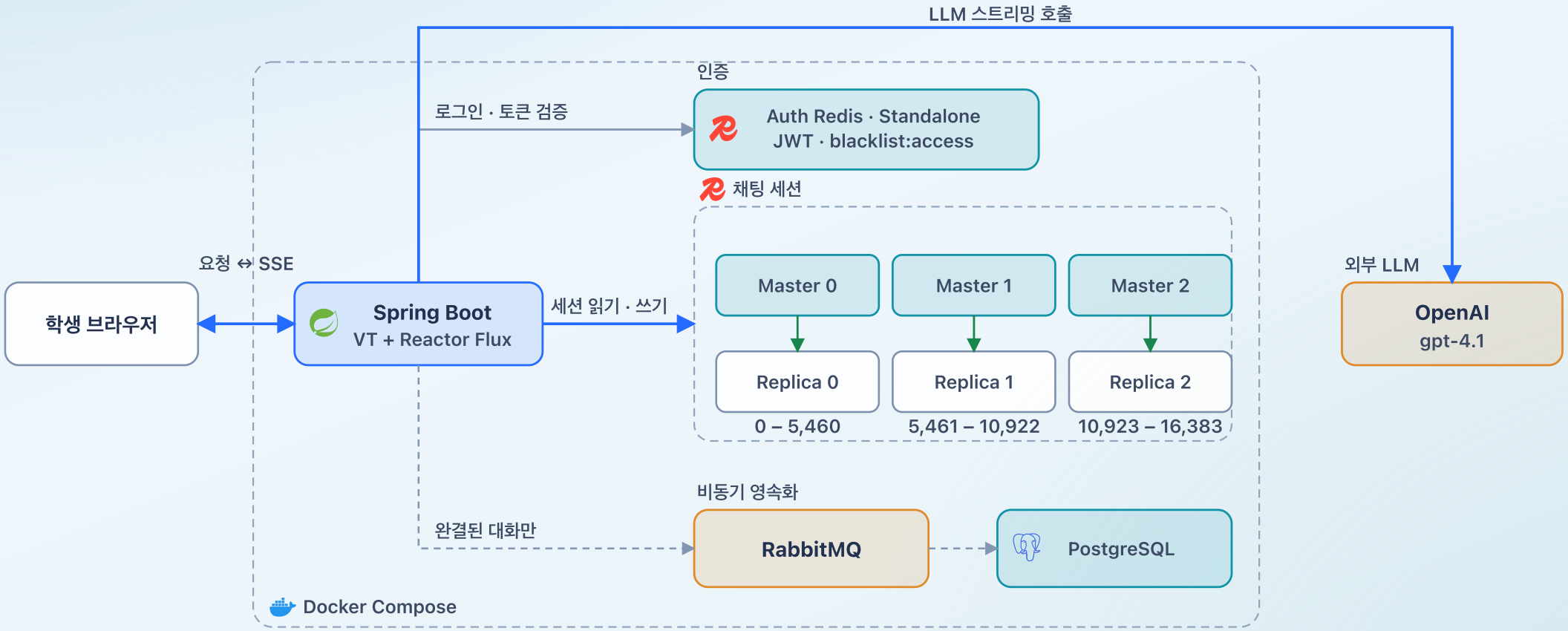
AI 튜터링 · Overview

AI 튜터링 서비스

2025-12 ~ 2026-03 · 9인 팀 백엔드 2인 중 백엔드 리드

9인 팀 백엔드 2인 중 백엔드 리드, 채팅 · 인증의 기술 선택과 구조 결정 주도. **Redis 를 용도별로 분리했다** —
인증 토큰은 Standalone 으로 충분하지만, 진행 중 세션은 유실되면 대화가 통째로 사라져 **Cluster 로 이중화**.

Spring · Spring AI · Project Reactor · SSE · RAG · Redis Cluster · RabbitMQ · Resilience4j · Spring State Machine · pgvector



14

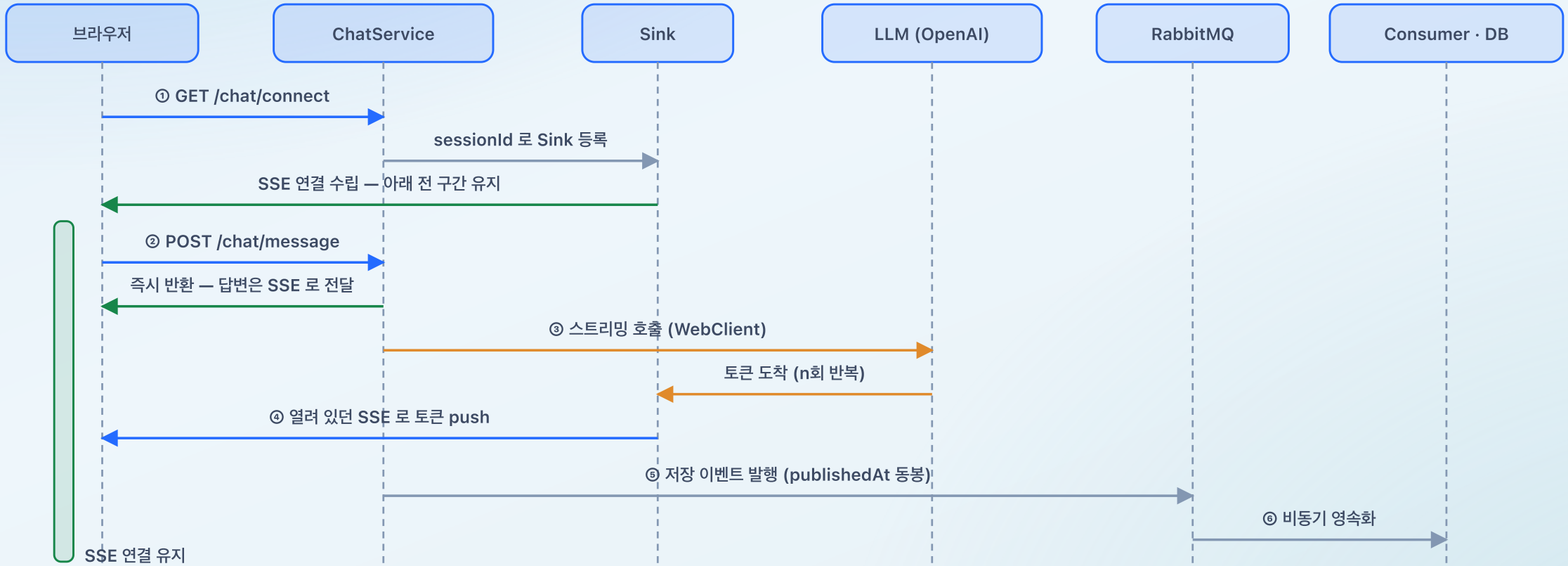
AI 튜터링 · 실시간 스트리밍 · Sequence

채팅 한 건이 흐르는 순서

연결 수립(①)부터 토큰 전달(④) · 영속화(⑥)까지

문제 동기 응답은 생성이 끝날 때까지 빈 화면 — 첫 글자 p95 **21.40초**(= 응답 완료), 그동안 요청 스레드 점유.

연결과 요청을 분리했다. ① 의 SSE 는 ⑥ 까지 열려 있고 ② 의 POST 는 **즉시 끝남** — 답변은 **열린 SSE 로** 나가고 저장(⑤⑥)은 RabbitMQ 뒤로 밀려 경로가 다름.



김민성 · Minsung Kim

15

AI 튜터링 · 실시간 스트리밍 · Pipeline

실시간 채팅 스트리밍 파이프라인

Virtual Thread 가 있는데 Reactor 를 더한 것은 취향이 아니라 API 제약 — 첫 글자 p95 12.37배 단축, 응답 완료 시각은 불변

문제 토큰을 `doOnComplete` 에서 한 번에 발행해 **첫 글자 = 응답 완료 시각**, 따로 `blockLast()` 가 **바운디드 풀 (core 10 / max 50 / queue 100)** 스레드를 스트림 끝까지 점유 — 원인이 **둘**이라 한 수단으로 풀리지 않음

해결 세 안을 나란히 놓고 **연결은 Virtual Thread, LLM 대기만 Reactor** 에 맡긴 하이브리드 채택 · 토큰은 도착 즉시 SSE 로 발행 — Spring AI 스트리밍(`ChatClient.stream()`)이 결과를 **Flux 로만** 주고, 전면 이전은 JPA · Security 재작성 비용

결과 첫 글자까지 p95 **21.40초 → 1.73초**(12.37배) · 스레드 풀 크기로 생기던 연결 상한(50) 제거 · 응답이 전부 끝나는 시각은 LLM 생성 속도가 정해 불변

세 안 비교 — 무엇을 풀고 어디서 막히는가

안	푸는 것	막히는 지점	이 하나로 둘 다 되나
WebFlux 전면 이전	연결과 대기를 모두 논블로킹으로	JPA · Security 를 리액티브로 **재작성**해야 함	**기각**
Virtual Thread 단독	설정 한 줄로 연결 수의 **스레드 풀 종속 제거**	Spring AI 스트리밍(`stream()`)은 결과를 **Flux 로만** 줌 — VT 만으로는 전달 방식이 그대로	**불충분**
하이브리드 (채택) 서블릿 + VT + Reactor	연결은 VT, **LLM 대기만** Reactor 에 위임	콜백 블로킹이 이벤트 루프를 점유 → `publishOn` 오프로딩 필요	**채택**

동기 응답 대비 스트리밍 전환 효과

측정 항목	개선 전	개선 후	변화
첫 글자까지 (p95)	21.40초	1.73초	**12.37배 단축**
첫 글자까지 (중앙값)	12.20초	1.09초	**11.19배 단축**
첫 글자까지 (RAG 검색 미경유, 중앙값)	11.65초	0.55초	**21.18배 단축**
동시 연결 상한	스레드 풀 최대 50, 초과 시 큐잉 후 거부	**풀 크기에 종속되지 않음**	**풀 상한 제거**
응답 완료까지 (중앙값)	12.20초	12.24초	불변, 개선 대상 아님

트레이드오프 스택이 둘로 섞여 **경계마다 스케줄러를 의식**해야 함 — 토큰 콜백의 블로킹(Redis 저장 · 상태 전이)을 그대로 두면 OpenAI 응답을 읽는 **이벤트 루프를 점유**해 같은 루프의 다른 사용자 스트림까지 멈춤. 두 경로 모두 구독 직후 · 콜백 앞에 `publishOn(boundedElastic)` 을 둬 해소

김민성 · Minsung Kim

16

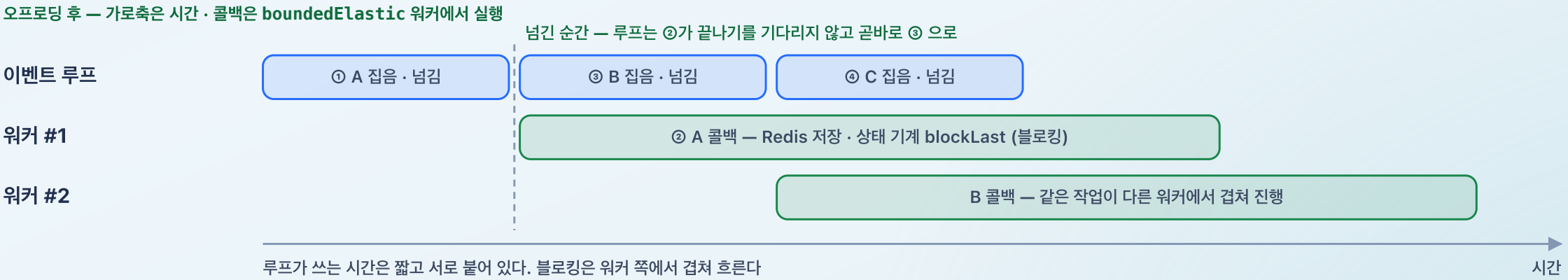
AI 튜터링 · 실시간 스트리밍 · Offloading

콜백이 이벤트 루프를 막을 때

한 사용자의 콜백 블로킹이 같은 루프가 맡은 다른 사용자 스트림까지 멈춘다

이벤트 루프는 여러 커넥션을 함께 맡아, 블로킹 콜백을 그 위에서 돌리면 **피해가 남의 요청까지 번진다**.
콜백 블로킹을 **300ms** 로 키우자 다른 세션 첫 청크가 **316ms** 밀림 — `publishOn` 뒤 **1~19ms**(5회).





김민성 · Minsung Kim

17

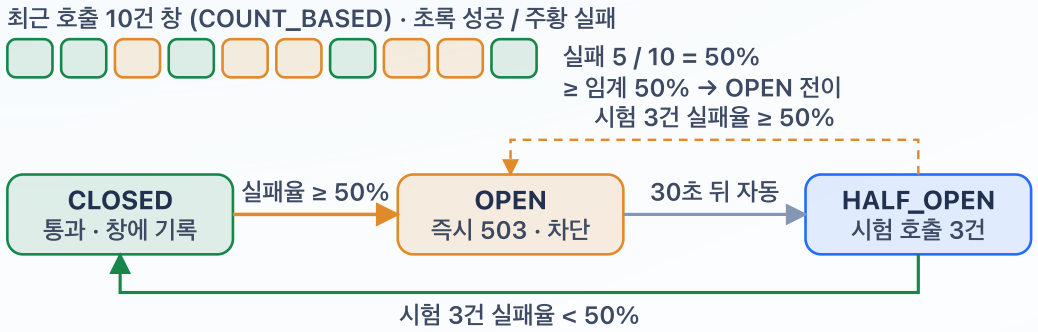
AI 튜터링 · 실시간 스트리밍 · Resilience

LLM 호출 · 세션 안정화 계층

장애 전파는 서킷으로 끊고, 비정상 전이는 막되 고착 상태는 회수한다

문제 외부 LLM 은 느려지거나 응답하지 않는다. **끊는 것만으로는 부족하고**, 끊긴 뒤 세션이 어느 상태에 멈춰 있는지까지 정해야 다음 요청이 받아들여진다. 그래서 **서킷(호출)과 상태 기계(세션)를 함께** 설계했다.

해결 ① 서킷 — 최근 호출 창의 실패율로 열고, 시험 호출로 닫는다



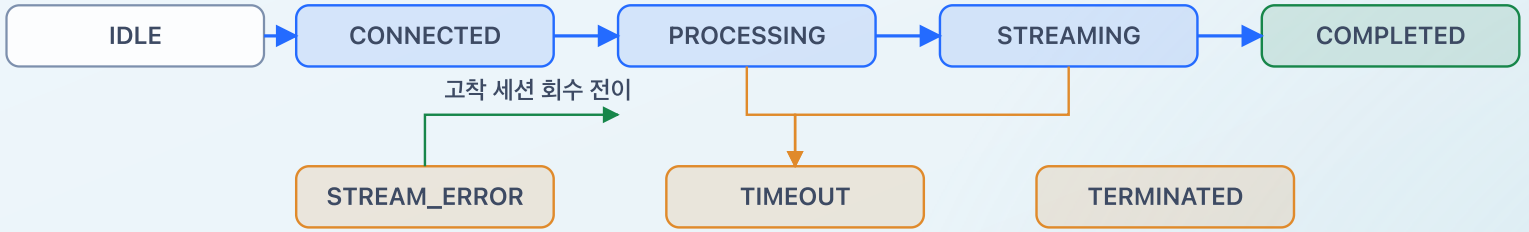
해결 ③ 서킷 범위를 구독 시점까지 넓힌다 — 반쪽 구간은 실패 주입으로 발견

`chatStream` 이 **메서드 본문에서** 히스토리 조회와 RAG 벡터 검색을 동기로 했다. Redis · 벡터 스토어가 죽으면 **Flux 를 만들기도 전에 예외가 나가는데**, 리액티브 서킷은 **구독 시점**부터 감싼다 — 그 실패는 **한 건도 집계되지 않았다.**

조회 · 조립을 `Flux.defer` 로 **구독 시점까지 옮겨** 검색 실패까지 서킷 안으로 편입. **실패 주입이 아니면 초록으로 남았을 결함.**

같은 결함의 두 번째 증상 — 에러 콜백을 건너뛰어 세션이 PROCESSING 에 고착, 첫 요청 500 뒤 2~6번째 연속 401. **감수한 비용** — `subscribe()` 전에는 아무 검증도 일어나지 않음

해결 ② 세션 상태는 선언된 (상태, 이벤트) 쌍으로만 — 빈 응답 · LLM 일시 실패로 PROCESSING 에 고착된 세션을 회수하는 전이를 따로 둠



선언된 (상태, 이벤트) 쌍만 허용 · 총 8상태 — 정상 5 + 에러 3

결과 시험용 엔드포인트(`/test/circuit` , LLM 과 같은 서킷 설정)에 3초 지연을 심은 부하 243,820건 중 95% 가 **1.76ms** 안에 응답(서킷의 즉시 거절) · 최댓값 3,026ms 는 서킷이 막기 전 3초를 그대로 기다린 호출

18

AI 튜터링 · 세션 저장소 · Redis Cluster

채팅 세션 저장소 설계

채팅만 Cluster 로 이중화, 읽기는 replica 로 분산해 포화 지점 이동

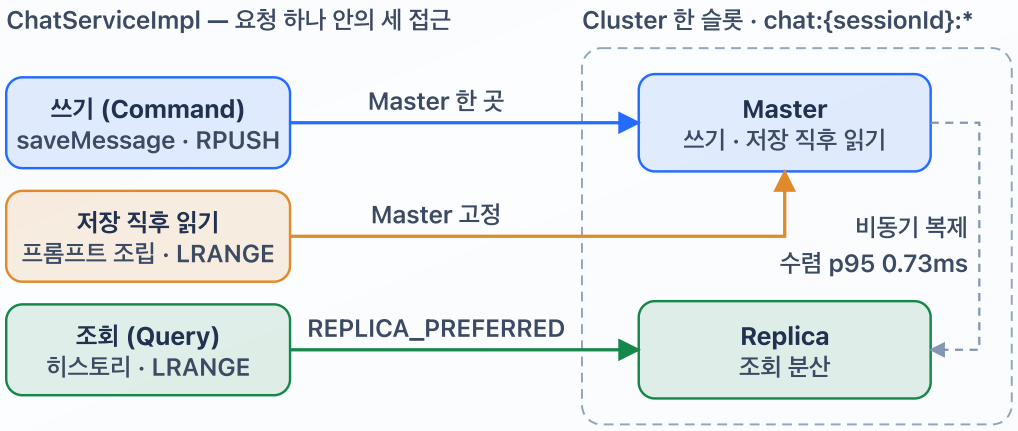
문제 Redis Standalone 단일 노드는 장애 시 **모든 세션의 멀티턴 문맥을 한 번에 잃는** 단일 장애점. **채팅 세션만 Cluster 로 이중화**(인증 토큰은 재로그인으로 복구돼 Standalone 유지)하고 읽기 · 쓰기 경로를 분리했다.

약 6만 → 약 11만
합계 처리량 포화 지점 이동
ops/s · 한 슬롯 · 최고점끼리 1.72배

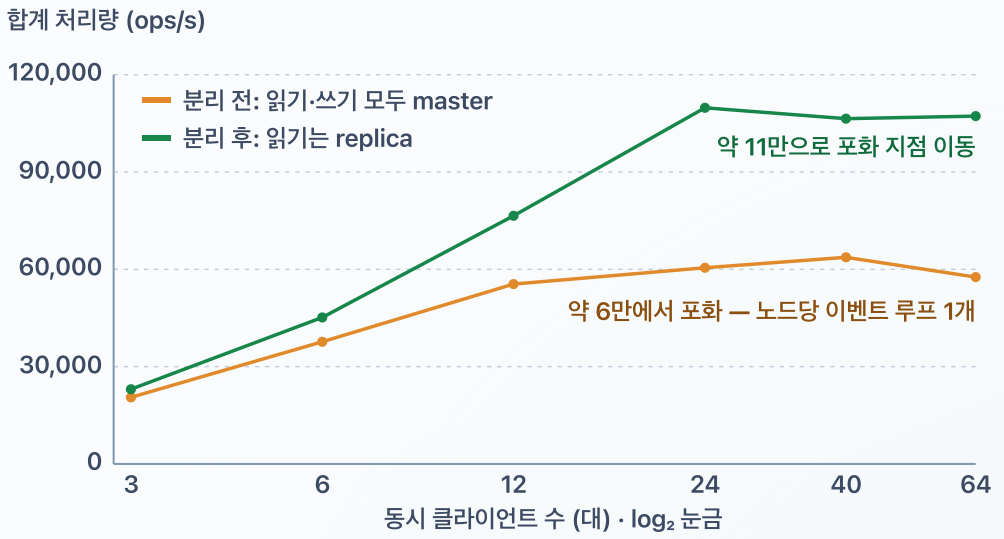
쓰기 +156%
가장 크게 이득 본 쪽
읽기는 +41% — 조회를 replica 로 빼자 쓰기 대기가 사라짐(24대)

7.85초
Master 강제 종료 → 쓰기 재개
3회 7.74 ~ 7.92초 · 장애 판정 5초 + 승격

해결 읽기 · 쓰기 경로 분리 — CQRS 식 라우팅(모델은 하나, 읽기만 replica)



결과 읽기/쓰기 분리 실측 — 합계 처리량



김민성 · Minsung Kim

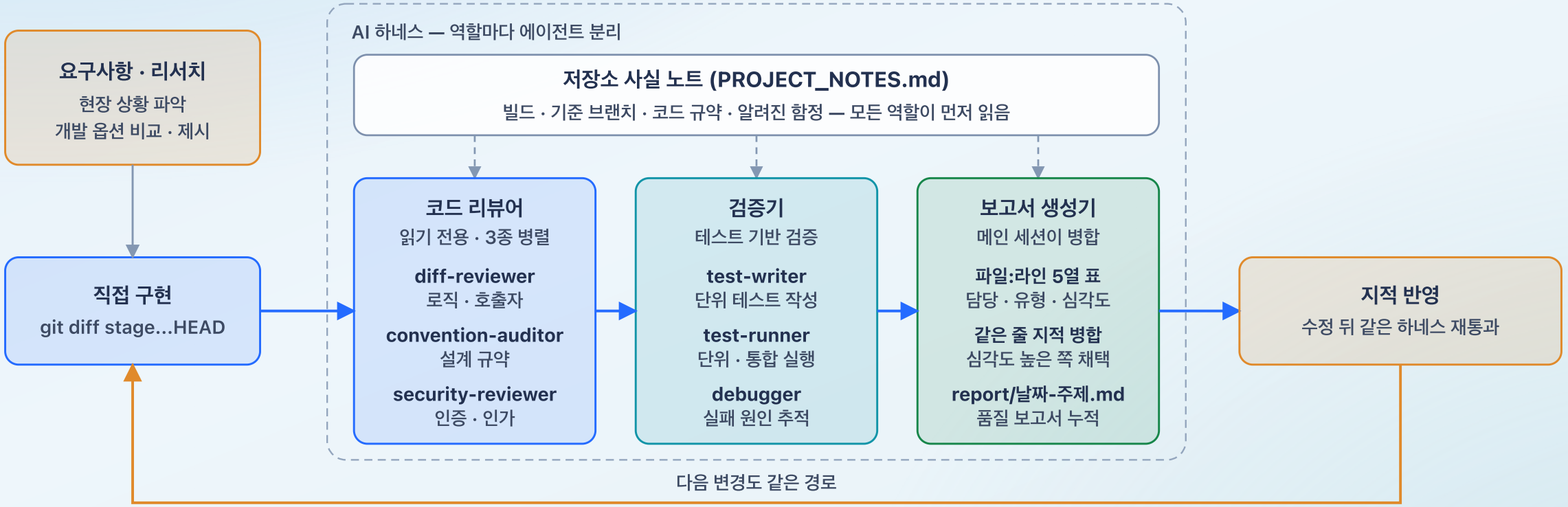
19

개발 방식 · AI Harness

AI 하네스로 코드 품질 검증

직접 구현한 코드를 리뷰어 · 검증기 · 보고서 생성기로 역할을 나눈 AI 에이전트에 통과

문제 AI 한 번의 호출에 리뷰 · 테스트 · 정리를 함께 맡기면 **지적이 한 관점에 몰리고**, 기준 브랜치 · Docker 가 필요한 통합 테스트 · 코드 규약 같은 **저장소 사실을 매번 다시 설명**해야 함.



결과 스트리밍 리팩터링 커밋 리뷰 — 리뷰어 3종 병렬 실행으로 **9건 검출(high 2 · medium 4 · low 3)**, 파일:라인 기준 한 표로 병합

THANK YOU

읽어주셔서 감사합니다.

Phone 010-8976-2340

Email mskim.bm@gmail.com

GitHub github.com/mskim98